

public void initializePieces()					
Function Description: Initializes Pieces on the board					
	Test Description	Sample Code	Expected Result	Actual Result	P/F
1	Initialize Elephant	Piece e1 = new Piece("Elephant", 8, 0, 2, player1, false);	Piece is instantiated	Piece is instantiated	P
	Piece	tiles[0][2].setPiece(e1);	Piece is set on tile	Piece is set on tile	
		player1.addPiece(e1);	Added to player's pieces	Added to player's pieces	
2	Initialize Lion	Panthera l1 = new Panthera("Lion", 7, 6, 0, player1, false);	Piece is instantiated	Piece is instantiated	P
	Piece	tiles[6][0].setPiece(l1);	Piece is set on tile	Piece is set on tile	
		player1.addPiece(l1);	Added to player's pieces	Added to player's pieces	
3	Initialize Rat	Rat r2 = new Rat("Rat", 1, 0, 6, player2, false);	Piece is instantiated	Piece is instantiated	P
	Piece	tiles[0][6].setPiece(r2);	Piece is set on tile	Piece is set on tile	
		player2.addPiece(r2);	Added to player's pieces	Added to player's pieces	
public boolean movePiece(Piece selectedPiece, String direction, Player currentPlayer)					
Function Description: moves selected piece on the board					
	Test Description	Sample Input	Expected Result	Actual Result	P/F
1	invalid tile	selectedPiece = Lion	boolean = false	boolean = false	P
		newTile = [0][-1]			
		currentTile = [0][0]			
2	valid tile	selectedPiece = Rat	boolean = true	boolean = true	P
		newTile = [2][3]			
		currentTile = [2][2]			
3	invalid animal	selectedPiece = Cat	boolean = false	boolean = false	P
		newTile = [2][2]			
		currentTile = [1][2]			
4	valid animal	selectedPiece = Rat	boolean = true	boolean = true	P
		newTile = [5][4]			
		currentTile = [5][5]			
public boolean canCapture(Tile currentTile, Tile newTile, String direction)					
Function Description: determines if a certain piece can capture the piece on the next tile					
	Test Description	Sample Input	Expected Result	Actual Result	P/F
1	invalid tile	currentTile = [0][0]	boolean = false	boolean = false	P
		newTile = [0][-1]			
		direction= W			
2	valid tile	currentTile = [1][2]	boolean = true	boolean = true	P
		newTile = [1][1]			
		direction= A			

	3	valid tile	currentTile = [4][2] newTile = [4][3] direction= S	boolean = true	boolean = true	P
	4	valid tile	currentTile = [6][3] newTile = [6][4] direction= D	boolean = true	boolean = true	P
public Tile getLandTile(Tile currentTile, Tile newTile , String direction)						
Function Description: validates and calculates the landing tile for the Lion and Tiger						
		Test Description	Sample Input	Expected Result	Actual Result	P/F
	1	valid tile	currentTile = [3][3] newTile = [0][3] direction= W	Tile = [0][3]	Tile = [0][3]	P
	2	valid tile	currentTile = [4][6] newTile = [4][2] direction= A	Tile = [4][2]	Tile = [4][2]	P
	3	invalid tile	currentTile = [3][3] newTile = [3][6] direction= S	Tile = [3][6]	Tile = [3][6]	P
	4	invalid tile	currentTile = [2][2] newTile = [2][6] direction= D	Tile = [2][6]	Tile = [2][6]	P
public boolean isValidMove(Tile currentTile, Tile newTile, Player currentPlayer, String direction)						
Function Description: checks if the chosen move by the player is allowed						
		Test Description	Sample Input	Expected Result	Actual Result	P/F
	1	invalid move (out of bounds)	currentTile = [0][0] newTile = [-1][0] currentPlayer= player1 direction= W	boolean = false	boolean = false	P
	2	invalid move (homebase of player1)	currentTile = [3][1] newTile = [3][0] currentPlayer= player1 direction= A	boolean = false	boolean = false	P
	3	invalid move (animal crossing lake (not rat))	currentTile = [2][2] newTile = [2][3] currentPlayer= player2 direction= D	boolean = false	boolean = false	P
	4	valid move (capturing a weaker	currentTile = [1][2] newTile = [2][2]	boolean = true	boolean = true	

	Test Description	Sample Input	Expected Result	Actual Result	P/F
1	correct input	input= 1	int= 1	int= 1	P
2	correct input	input=2	int= 2	boolean = true	P
3	incorrect input	input=0	int= error	int= error	P
public void gameLoop()					
Function Description: loops the game proper until there is a winner					
	Test Description	Sample Input	Expected Result	Actual Result	P/F
1	No winner	gameOver = false	Game continues	Game continues	P
2	Player 1 wins	gameOver = true	Game stops	Game stops	P
3	Player 2 wins	gameOver = true	Game stops	Game stops	P
public Piece getAnimalPiece()					
Function Description: check if input is a valid symbol and returns chosen piece					
	Test Description	Sample Input	Expected Result	Actual Result	P/F
1	correct input	piece = E	piece = Elephant	piece = Elephant	P
2	correct input	piece = L	piece = Lion	piece = Lion	P
3	incorrect input	piece = H	piece = error	piece = error	P
public String getDirection()					
Function Description: gets direction of movement of the animal piece from player					
	Test Description	Sample Input	Expected Result	Actual Result	P/F
1	Up	direction = W	Returns "W"	Returns "W"	P
2	Down	direction = S	Returns "S"	Returns "S"	P
3	Invalid	direction = F	Asks user to input again	Asks user to input again	P
public int getAnimalRandomizerInput()					
Function Description: validates and returns chosen animal					
	Test Description	Sample Input	Expected Result	Actual Result	P/F
1	correct input	int = 1	int = 1	int = 1	P
2	correct input	int = 2	int = 2	int = 2	P
3	incorrect input	int = 0	int = error	int = error	P
public void randomizeAnimals()					
Function Description: randomizes animals					
	Test Description	Sample Input	Expected Result	Actual Result	P/F
1	Test 1	Method is called	Randomized Animals	randomAnimals = [E, P, C, R, L, W, D, T]	P
2	Test 2	Method is called	Randomized Animals	randomAnimals = [W, D, T, R, P, E, L, C]	P
3	Test 3	Method is called	Randomized Animals	randomAnimals = [T, C, L, W, D, R, P, E]	P

	public void switchTurns()				
	Function Description: switches the turns of players				
		Test Description	Sample Input	Expected Result	Actual Result
	1	Player 1 done	Method is called	currentPlayer = player2	currentPlayer = player2
	2	Player 2 done	Method is called	currentPlayer = player1	currentPlayer = player1
	public boolean checkWinningCondition()				
	Function Description: randomizes animals				
		Test Description	Sample Input	Expected Result	Actual Result
	1	Player 1 wins	Player 1 is in the opponent's homebase	Prints out player1 wins	Prints out player1 wins
	2	Player 2 wins	Player 2 is in the opponent's homebase	Prints out player2 wins	Prints out player2 wins
	3	Player 1 no more pieces	Player 1's pieces list is empty	Prints out player2 wins	Prints out player2 wins
	public void playAgain()				
	Function Description: asks user if they want to play again at the end of a game				
		Test Description	Sample Input	Expected Result	Actual Result
	1	Yes	Y	Game resets	Game resets
	2	No	N	System exit	System exit
	3	Other input	E	System exit	System exit